

Offset Accounting — Design and Algorithm

Subject: module 2 of the consensus library, `kgl_consensus_offset.{h,cpp}` (deepseek-refactor), as installed in `KGL_Gene/kgl_genomics/kgl_consensus/`. **Companion:** `consensus_algorithm.md` (whole pipeline), `deepseek-refactor.md` (plan). **Status:** final; verified by the behavioural suite, the module compile gates and a 1422-genome production run.

1. Purpose

Offset accounting answers one question and nothing else:

Given a reference (contig) coordinate, where does it appear in the modified sequence after every resolved edit has been applied?

The module is deliberately **pure integer interval mathematics**:

- it includes no sequence type and never touches `DNA5SequenceLinear`;
- it holds no reference bases, no output buffer and no mutable state after `build`;
- it can be built, copied, shared across threads and queried without materialising any consensus sequence at all.

Its intended uses:

1. slice a feature interval (exon, domain, buffer) from a modified sequence;
2. annotate or liftover an interval vector without building a sequence (`modifyIntervals`);
3. provide the coordinate basis that the sequence materialiser (`buildConsensus`) uses, so that mapping and content are projections of the same schedule;
4. expose the inverse mapping (modified $\rightarrow$ reference) for reporting, where it is defined.

2. Position in the architecture

```
Module 1  selectVariants / selectWindowVariants    → SelectedVariants (ContigDB)
Module 2  resolveVariants                        → AppliedVariants { vector<Edit> }
          OffsetAccounting::build                → prefix-sum tables
          toModified / toReference / shift / isDeleted / modifyIntervals
Module 3  buildConsensus(reference, origin, accounting) → ConsensusSequence
```

Module 2 is the **only** owner of the edit schedule. Module 3 consumes the same `OffsetAccounting` rather than re-resolving variants, which is what guarantees that the interval chain and the materialised sequence cannot disagree.

Data flow inside module 2:

```

ContigDB (canonical, one per locus, policy-filtered, shadow-pruned by module 1)
|
└─ resolveVariants
Edit list: canonicalised, sorted by begin, deletes made disjoint, shadows pruned
|
└─ OffsetAccounting::build
edits_[]          sorted, disjoint
shift_before_[]   cumulative Δ before each edit
modified_begin_[] toModified(edit.begin) for each edit
total_adjust_     final cumulative Δ

```

3. The edit model

```

enum class EditKind : std::uint8_t { Snp, Insert, Delete };

struct Edit {
    EditKind kind{EditKind::Snp};
    ContigOffset_t begin{0}; // first consumed reference base; insertion point for inserts
    ContigOffset_t end{0};   // one past the last consumed base; == begin for inserts
    std::shared_ptr<const Variant> variant;
};

```

Constructed from `Variant::modifyInterval()`, which returns the canonical span with the shared anchor base excluded:

Kind	begin	end	Consumed	Inserted	Payload
SNP	<code>offset()</code>	<code>begin + 1</code>	1	1 (replacement)	alternate base
Delete	<code>modify.lower() = offset()+1</code>	<code>modify.upper() = begin+n</code>	n	0	none
Insert	<code>modify.lower() = offset()+1</code> (insertion point)	<code>begin</code>	0	n	alternate minus anchor

Derived operations:

```

consumed() = end - begin
inserted() = (Insert: alternateSize - referenceSize; Snp: 1; Delete: 0)
net Δ      = inserted() - consumed()

```

Key conceptual point: an insert consumes **no** reference coordinate. It is a point event between reference bases `i0-1` and `i0`, which is why `end == begin` and why the module's boundary conventions below are all about comparing a query offset to the insertion point.

4. `resolveVariants` — building the schedule

Input `SelectedVariants` (output of module 1), optional `DeleteOverlapPolicy`.

4.1 Canonicity gate

Every variant must be canonical (`1X`, `1MnD`, `1MnI`). A non-canonical edit is an error (`VariantNotCanonical`), logged with the variant's HGVS; the module never applies a variant it cannot interpret. (Module 1 normally prevents this.)

4.2 Sort and partition

Edits are sorted by `begin`, then split into deletes and non-deletes.

4.3 Delete disjointness

Deletes are processed in order; consecutive deletes that overlap or touch are resolved by policy:

Policy	Behaviour
Union (default)	extend the retained delete to the union of the two spans; the retained record adopts the later variant
Drop	discard the later (downstream) delete; the upstream one wins
Error	fail with <code>DeletesOverlap</code>

Result: `disjoint_deletes[]` with strictly increasing, non-overlapping spans.

Why deletes first: a delete removes reference material and any non-delete inside it ceases to exist. Resolving the deleted set before pruning shadows is what makes the shadow test exact and order-independent.

4.4 Shadow pruning

A non-delete edit is **discarded** if its member span intersects any disjoint delete. The member span is the set of reference positions the variant claims to affect:

```
SNP, Delete : [begin, end)
Insert      : [begin - 1, begin + 1)    (anchor base and insertion point)
```

Consequences:

- a SNP at a delete anchor ($o = d_0 - 1$) survives; a SNP at `d0` is pruned;
- the first surviving SNP after a delete is at $o = d_1$;
- an insert is pruned iff its insertion point $i_0 \in [d_0, d_1]$ (both boundaries included: immediately before the first deleted base or immediately after the last are treated as consumed with the deleted material);
- discarded edits increment `stats.upstream_deleted_`.

4.5 One edit per locus

After pruning and re-sorting, two edits sharing a `begin` are an error (`VariantDuplicateLocus`). Module 1's per-locus resolution normally guarantees this; the check is defensive and protects the prefix-sum arithmetic.

4.6 Post-conditions

```
edits sorted by begin, strictly: edits[i].begin >= edits[i-1].end
one edit per begin
```

These are the invariants `OffsetAccounting::build` re-checks defensively.

5. `OffsetAccounting::build` — the prefix sums

One linear pass produces three parallel arrays:

```
cumulative = 0
for each edit e_i:
    shift_before[i] = cumulative
    modified_begin[i] = e_i.begin + cumulative
    cumulative += e_i.inserted() - e_i.consumed()
total_adjust_ = cumulative
```

Array	Meaning
<code>shift_before[i]</code>	net Δ of all edits strictly before <code>e_i</code>
<code>modified_begin[i]</code>	where <code>e_i</code> starts in the modified coordinate space
<code>total_adjust_</code>	net length change of the whole schedule

Build fails with `DeletesOverlap` if the schedule is not sorted and disjoint (defensive; `resolveVariants` guarantees it).

The tables are immutable after build. `OffsetAccounting` is therefore a value type that can be shared across threads and copied cheaply relative to the edit list.

5.1 Why prefix sums rather than recomputation

The reference implementation maintained a mutable `ModifiedOffsetMap` populated while the sequence was being modified, with a cumulative adjustment looked up via a **strictly-less predecessor search** over stored records. That design had three consequences, all removed here:

1. the lookup could not see an edit at the query offset itself, producing the adjacent-anchor insert misplacement (payload B placed inside payload A);
2. the delete shadow span had to be reconstructed from a stored key and a length, which is the D1 bug (the key is the region start, not the delete start, for a partial-low delete);
3. mapping and materialisation were two independent computations that could diverge.

With the prefix sums computed once from an authoritative edit list:

- mapping is arithmetic against explicit edit spans, so the shadow test uses the true deleted interval;
- there is no stored-key reconstruction;
- `buildConsensus` consumes the identical schedule, so the chain and the sequence are consistent by construction.

6. Queries

Let `edits_` be the sorted disjoint schedule, `prefix[i] = shift_before[i]`, `$\Delta_i = \text{inserted}_i - \text{consumed}_i$` .

6.1 `toModified(offset)` — reference → modified

```
count = number of edits with begin <= x      (binary search)
if count == 0: return x
e = edits_[count-1]; i = count-1

if e is a Delete and x < e.end:
    return modified_begin_[i]                // collapse to the deletion boundary

if e is an Insert and x == e.begin:
    return x + prefix[i]                    // payload start: payload NOT yet counted

return x + prefix[i] + Δi
```

Three rules, in order:

1. **Before all edits** — identity.
2. **Inside a deleted span** — collapse to the deletion boundary `modified_begin_[i]`. This is total and monotone by design: deleted reference coordinates have no modified image, and collapsing (rather than failing) lets callers slice across deleted regions and receive empty or clipped results.
3. **Insert boundary convention** — a query exactly at the insertion point `i0` returns the **payload start**, i.e. the payload is *not yet* included: - an interval *beginning* at `i0` therefore includes the payload (its image is `[i0+prefix, upper]`, and the lower endpoint sits before the payload); - an interval *ending* at `i0` excludes the payload (its upper endpoint sits before it).

After the point, the accumulated `Δi` applies and every later offset is shifted.

Why the insert convention matters. It is shared verbatim with `ConsensusSequence::slice` and matches the reference implementation's strictly-less predecessor lookup: an interval starting at the insertion point absorbs the inserted bases, an interval ending there does not. Module 2 originally counted the payload at `x == i0` (returning `x + prefix + Δ`), which disagreed with the materialiser at that single coordinate. The two modules are now bit-identical at every boundary; the probe below demonstrates it.

6.2 `toModified(interval)`

```
[a, b) → [toModified(a), toModified(b))
```

The image of a right-open interval under a monotone map is the interval of its endpoint images. One rule produces every case:

Reference interval vs edits	Image	Reading
before all edits	unchanged	identity
after upstream edits	shifted by cumulative Δ	liftover
contains <code>i0</code> and extends past it	expanded by payload	insert absorbed
contains a delete span	contracted	deleted bases removed
wholly inside a delete	<code>[p, p)</code> empty	no modified material
overlapping an upstream delete	lower collapses/clips to the boundary	partial overlap

6.3 `shift(offset)`

`toModified(x) - x` as a signed value; negative in net-deleted regions. Convenience for reporting and diagnostics.

6.4 isDeleted(offset)

True iff the last edit at or before `x` is a delete and `x < e.end`. Cheap predicate for filtering; equivalent to an empty image.

6.5 toReference(modified_offset) — modified → reference (partial)

```
count = number of edits with modified_begin <= m      (binary search, same tables)
if count == 0: return m
e = edits_[count-1]; i = count-1

if e is an Insert and m < modified_begin[i] + inserted_i:
    return nullopt                                     // no reference base in a payload

return m - prefix[i] - Δ_i
```

Defined everywhere except **inside an inserted payload** — inserted bases have no reference origin. At a deletion boundary the first retained reference base (the delete end) is returned. This partiality is deliberate and is expressed in the return type rather than an error code.

6.6 modifyIntervals(accounting, intervals)

Maps a vector of reference intervals in one call, in the same order, using `toModified(interval)`. Zero sequence work; the canonical entry point for feature annotation and liftover against the same edit schedule used by the materialiser.

7. Boundary semantics — the complete table

Take an insert with insertion point `K` and payload length `n = 2` (payload `GG`), and a delete `D = [d0, d1]`. Reference bases ... `base(K-1)`, `base(K)`, `base(K+1)` ...

7.1 Insert

Query	toModified	slice	Interpretation
<code>K - 1</code>	<code>K - 1 + prefix</code>	local <code>x - W0</code> before payload	last base before the point
<code>K</code>	<code>K + prefix</code>	payload start	payload not yet counted
<code>K + 1</code>	<code>K + 1 + prefix + n</code>	payload end + 1	payload counted
interval <code>[K-1, K)</code>	<code>[K-1+p, K+p)</code>	base(K-1) only	payload excluded (ends at <code>K</code>)
interval <code>[K, K+1)</code>	<code>[K+p, K+1+p+n)</code>	payload + base(K)	payload included (begins at <code>K</code>)
interval <code>[K-1, K+1)</code>	<code>[K-1+p, K+1+p+n)</code>	base(K-1) + payload + base(K)	both sides
interval <code>[K, K)</code>	empty	empty	point interval

7.2 Delete

Query	toModified	slice	Interpretation
<code>x < d0</code>	<code>x + prefix</code>	before the span	untouched
<code>x ∈ [d0, d1)</code>	<code>d0 + prefix</code> (boundary)	collapses to boundary	deleted

Query	toModified	slice	Interpretation
x = d1	d1 + prefix + net	first retained base	retained
interval inside the span	empty image	empty sequence	wholly deleted
interval straddling the span	contracted image	clipped sequence	partial

7.3 Module agreement probe

Insert at anchor 99 → insertion point $K = 100$, payload GG, reference 300 bases:

```

Module 2 (OffsetAccounting)
toModified(99) = 99
toModified(100) = 100
toModified(101) = 103
[99,100) -> [99,100) len=1
[100,101) -> [100,103) len=3
[99,101) -> [99,103) len=4

Module 3 (ConsensusSequence::slice)
[99,100) len=1 seq=G
[100,101) len=3 seq=GGC
[99,101) len=4 seq=GGGC

```

Both modules agree at every boundary, including the payload start and the post-payload shift.
Reference bases: base99=G, base100=C, base101=A.

8. Worked example — the full chain

Schedule: insert +5 at point 11, delete [20,23), SNP 30, delete [40,50), insert +2 at point 61. The prefix sums are:

i	edit	begin	end	Δ	shift_before	modified_begin
0	Insert +5	11	11	+5	0	11
1	Delete	20	23	-3	+5	25
2	SNP	30	31	0	+2	32
3	Delete	40	50	-10	+2	42
4	Insert +2	61	61	+2	-8	53

total_adjust = -6.

Query results:

Query	Result	Derivation
toModified(0)	0	count 0 → identity
toModified(10)	10	before all edits
toModified(11)	11	insertion point → payload start
toModified(12)	17	past the point: 12 + 0 + 5
toModified(19)	24	19 + 5
toModified(20)	25	inside delete → collapse to modified_begin
toModified(22)	25	same
toModified(23)	25	delete end (x < end false) but Δ counted: 23 + 5 - 3
toModified(30)	32	SNP, neutral: 30 + 5 - 3 + 0
toModified(40)	42	delete boundary
toModified(50)	42	50 + 2 - 10

Query	Result	Derivation
<code>toModified(61)</code>	53	insertion point: $61 + (-8)$
<code>toModified(62)</code>	56	$62 - 8 + 2$
<code>toModified(200)</code>	194	$200 - 6$

Interval vector `modifyIntervals`:

Input	Output	Reading
<code>[0, 10)</code>	<code>[0, 10)</code>	upstream of all edits
<code>[5, 11)</code>	<code>[5, 11)</code>	ends at insertion point: payload excluded
<code>[11, 12)</code>	<code>[11, 17)</code>	begins at insertion point: payload + one base
<code>[20, 23)</code>	<code>[25, 25)</code>	wholly deleted
<code>[20, 25)</code>	<code>[25, 27)</code>	clipped at the delete, two retained bases
<code>[55, 65)</code>	<code>[47, 59)</code>	10 retained + 2 inserted

9. Correctness properties

- Totality.** `toModified(offset)` and the interval overload are defined for every input; deleted coordinates collapse rather than fail.
- Monotonicity.** $x \leq y \Rightarrow \text{toModified}(x) \leq \text{toModified}(y)$.
- Length identity per region.** For an interval containing a set of edits, $|\text{image}| = |\text{input}| + \sum_{\text{applied}} \Delta$, with deleted spans contributing the negative of the intersected length.
- Boundary consistency.** `toModified` and `ConsensusSequence::slice` agree at every coordinate, including insert points, payload ends and delete boundaries (§7.3).
- Partial inverse correctness.** For every modified offset not inside a payload, `toReference(toModified(x)) == x`; for inserted payloads `toReference` is `nullopt` rather than a wrong answer.
- Order independence of disjoint edits.** The prefix sum is a function of the edit multiset, so resolving the same variants in the same canonical order always yields the same map.
- No stored-key dependence.** Deletion membership uses `[edit.begin, edit.end)` directly. There is no reconstruction from a record key, so the D1 overshoot cannot occur.

10. Complexity

Operation	Cost
<code>resolveVariants</code>	$O(V \log V)$ (sort) + $O(V \log D)$ shadow pruning
<code>OffsetAccounting::build</code>	$O(V)$
<code>toModified</code> / <code>toReference</code> / <code>isDeleted</code>	$O(\log V)$ binary search
<code>modifyIntervals</code>	$O(k \log V)$ for k intervals
Memory	$O(V)$ (three parallel arrays)

V is the number of edits in the window; k the number of intervals mapped.

11. Design decisions and rationale

Decision	Rationale
Pure interval module, no sequence dependency	interval mapping is independently usable (liftover/annotation) and testable without constructing DNA
Immutable after build	safe to share across threads; no mutable cumulative state to corrupt
Prefix sums over recomputation	<code>O(log V)</code> queries and a single authoritative mapping basis
Deletes resolved before shadow pruning	makes pruning exact and independent of edit order
Collapse, do not error, inside deletes	callers slice across deleted regions and need clipped/empty results, not failures
Partial inverse via <code>std::optional</code>	inside payloads there is no reference origin; the type says so
Single boundary convention shared with module 3	avoids two divergent mappings of the same schedule
Errors as <code>std::expected</code>	<code>resolveVariants / build</code> can fail cleanly; queries cannot fail

12. Defects this design removes

Reference defect	Mechanism in the reference	Remedy here
D1 delete shadow overshoot	span rebuilt as <code>[stored key, stored key + updating length)</code> ; for a partial-low delete the key is the region start, so retained bases were wrongly collapsed	membership tested against the true <code>[begin, end)</code>
Adjacent-anchor insert mis-placement	strictly-less predecessor lookup could not see the edit at the query offset, placing payload B inside payload A	explicit <code>Insert && x == begin</code> rule; payloads are point events in the schedule
Dual accounting basis	full-span translation for offset vs intersected span for reconciliation	one schedule, one prefix sum, one basis
Mapping/materialiser divergence	two independent computations	both consume the same <code>OffsetAccounting</code>
Undetected malformed schedules	no structural checks	<code>build</code> verifies sorted/disjoint; resolution verifies one-per-locus

13. Failure modes

Error	Trigger
<code>NullContig</code>	null input contig in <code>resolveVariants</code>
<code>VariantNotCanonical</code>	a variant that is not <code>1X / 1MnD / 1MnI</code>
<code>VariantDuplicateLocus</code>	two surviving edits with the same <code>begin</code>
<code>DeletesOverlap</code>	overlapping deletes under the <code>Error</code> policy, or a non-disjoint schedule at build

All are returned as `std::expected`; the caller (module 3 or the transcript facade) logs once and abandons the region. Queries themselves never fail — the only partial result is `toReference` inside payloads, expressed as `std::nullopt`.

14. Verification

Covered by `kgl_consensus_test.cpp`:

- the \$8 chain: every `toModified` value, `totalAdjust`, and the interval vector;
- delete collapse (`[20,23)` → `[25,25)`);
- insert boundaries (`[5,11)` excludes, `[11,12)` includes the payload);
- the module-agreement property at an insertion point (\$7.3 probe);
- the D1 proof via the consensus layer (`[55,60)` maps to reference `[55,60)` for region `[50,100)` , delete `[41,51)`);
- whole-sequence mapping (`[0,200)` → `[0,194)` for the chain).

The production run additionally exercised the module across 1422 genomes and 5633 transcripts; the interval chain agreed with the materialised sequence throughout (the transcript validity totals match the reference implementation after the D1 and insert-boundary corrections).

Appendix A — API summary

```
enum class EditKind : std::uint8_t { Snp, Insert, Delete };
enum class DeleteOverlapPolicy : std::uint8_t { Union, Drop, Error };

struct Edit {
    EditKind kind; ContigOffset_t begin, end;
    std::shared_ptr<const Variant> variant;
    ContigSize_t consumed() const noexcept;
    ContigSize_t inserted() const noexcept;
    std::string insertedPayload() const;
};

struct AppliedVariants { ContigId_t contig_id; std::vector<Edit> edits; FilteredVariantStats stats; };

std::expected<AppliedVariants, ConsensusError>
resolveVariants(const SelectedVariants&, DeleteOverlapPolicy = Union);

class OffsetAccounting {
    static std::expected<OffsetAccounting, ConsensusError> build(const AppliedVariants&);
    ContigOffset_t toModified(ContigOffset_t) const noexcept;
    OpenRightUnsigned toModified(const OpenRightUnsigned&) const noexcept;
    SignedOffset_t shift(ContigOffset_t) const noexcept;
    bool isDeleted(ContigOffset_t) const noexcept;
    std::optional<ContigOffset_t> toReference(ContigOffset_t) const noexcept;
    SignedOffset_t totalAdjust() const noexcept;
    std::span<const Edit> edits() const noexcept;
    bool empty() const noexcept;
};

std::vector<OpenRightUnsigned>
modifyIntervals(const OffsetAccounting&, std::span<const OpenRightUnsigned>);
```

Appendix B — Boundary rule summary

Situation	Rule
Query before all edits	identity
Query at/inside a delete	collapse to <code>modified_begin</code>
Query at an insertion point	payload start (payload not yet counted)

Situation	Rule
Query past an insertion point	add the payload via Δ
Interval beginning at an insertion point	payload included
Interval ending at an insertion point	payload excluded
Interval wholly inside a delete	empty image
Modified offset inside a payload (toReference)	nullopt